

Multi-Instance Security Degradation of Code-Based KEMs

Alexander May  and Gabriel Sá Diogo 

Ruhr-University Bochum, Bochum, Germany
`{alex.may, Gabriel.SaDiogo}@rub.de`

Abstract. The security of most prominent code-based key encapsulation mechanisms (KEMs) relies on the hardness of the syndrome decoding problem. It is well-known that in the presence of n syndromes, one gets a speed-up of roughly $\sqrt{n}$ for decoding a single syndrome by a technique called *Decoding One Out of Many* (DOOM), due to Sendrier. Modern code-based schemes like HQC and BIKE work over a polynomial ring $\mathbb{F}_2[X]/(X^n - 1)$ that naturally leads to n syndromes. As a consequence, DOOM-type speed-ups of $\sqrt{n}$ have been taken into account for the HQC and BIKE parameter selection in the single-instance setting.

However, we analyse a naturally appearing *multi-instance* setting, where the same public key is used to derive M session keys $K^{(1)}, \dots, K^{(M)}$. Our attack goal is to reconstruct a single session key $K^{(i)}$.

We show that in a BIKE multi-instance setting an attacker can construct a DOOM instance with nM syndromes. In an HQC and Classic McEliece multi-instance setting, an attacker obtains M syndromes. Our results show that multi-instance security of code-based KEMs degrades as a function of M . For KEMs designed for NIST security level 1 we drop below the desired 143 bits for a number of session keys $M \geq 2^{34}$ (HQC-1), $M \geq 2^{11}$ (BIKE-1), respectively $M \geq 2^{21}$ (mcecliece3488-64). As a conclusion, the public keys of all three code-based KEMs should be updated regularly.

Keywords: Multi-Instance Security · HQC · BIKE · Classic McEliece · Decoding One Out of Many

1 Introduction

In this work we consider the three code-based key encapsulations mechanisms (KEMs) HQC, BIKE and Classic McEliece [GAMA⁺25, ABB⁺24, BCC⁺22]. These KEMs reached the fourth round of the NIST PQC Standardization Process [Nat], with HQC being selected in early 2025 as an upcoming standard. All three KEMs rely on the hardness of syndrome decoding. In the syndrome decoding problem (over $\mathbb{F}_2$), one gets as input a parity-check matrix $P \in \mathbb{F}_2^{(n-k) \times n}$ of some linear $[n, k]$ -code and a syndrome $\mathbf{s} \in \mathbb{F}_2^{n-k}$. The goal is to find an error vector $\mathbf{e} \in \mathbb{F}_2^n$ of known small Hamming weight w such that $P\mathbf{e} = \mathbf{s}$.

Decoding One Out of Many (DOOM). In a DOOM scenario, one obtains multiple syndrome decoding instances $(P, \mathbf{s}^{(i)})_{1 \leq i \leq m}$ with the same parity-check matrix P . The goal is to find a single error vector $\mathbf{e}^{(i)}$ such that $P\mathbf{e}^{(i)} = \mathbf{s}^{(i)}$.

Modern decoding algorithms use a combination of permutation and enumeration. On the one hand, the presence of m possible target syndromes $\mathbf{s}^{(1)}, \dots, \mathbf{s}^{(m)}$ speeds up the search for a good permutation by a factor of roughly m . On the other hand, the matching to m targets slows down the enumeration by roughly $\sqrt{m}$. This leads in total to a speedup of $O(\sqrt{m})$ that was shown for Dumer-Stern by Sendrier [Sen11], and for MMT by Esser, May and Zweyding [EMZ22].

The Quasi-Cyclic Structure of HQC and BIKE. HQC and BIKE both work over the ring $R := \mathbb{F}_2[X]/(X^n - 1)$ which corresponds naturally to the vector space $\mathbb{F}_2^n$. For instance, an HQC public key is $(\mathbf{h}, \mathbf{s}) \in R^2$, and the secret key consists of weight- w vectors $\mathbf{x}, \mathbf{y} \in R$ such that

$$\mathbf{s} = \mathbf{x} + \mathbf{h} \cdot \mathbf{y}. \quad (1)$$

From $\mathbf{h}$ we define a parity-check matrix $P = [I_n \mid \text{rot}(\mathbf{h})] \in \mathbb{F}_2^{n \times 2n}$, where $\text{rot}(\mathbf{h}) \in \mathbb{F}_2^{n \times n}$ consists of n rotations of $\mathbf{h}$. Thus, the HQC key relation from Eq.(1) translates to a syndrome decoding instance $(P, \mathbf{s})$ with weight- $2w$ error vector $\mathbf{e} = (\mathbf{x}, \mathbf{y}) \in \mathbb{F}_2^{2n}$. Note that from Eq.(1) we also have

$$X^j \cdot \mathbf{s} = X^j \cdot \mathbf{x} + \mathbf{h} \cdot X^j \cdot \mathbf{y} \in R \quad \text{for } 0 \leq j < n.$$

Thus, the ring structure automatically yields n syndrome decoding instances $(P, \mathbf{s}^{(j)}) = (P, X^j \cdot \mathbf{s})$ with error vectors $\mathbf{e}^{(j)} = (X^j \cdot \mathbf{x}, X^j \cdot \mathbf{y})$.

1.1 Our Results.

Multi-Instance Security. We study a *multi-instance* attack scenario, where an attacker targets M sessions of a KEM under the same public key, resulting in M session keys $K^{(1)}, \dots, K^{(m)}$. Our goal is to recover a single session key $K^{(i)}$. Ideally, such a 1-out-of- M session key recovery should not be significantly easier than attacking a single instance, if at all. Unfortunately, we show that the multi-instance security of all code-based KEMs studied in this work degrades with roughly $\sqrt{M}$.

HQC: 1-out-of- M Session Key Recovery. HQC derives its session keys K from some random message m , among other publicly known things, via a key derivation function KDF. The message m itself is encrypted as a ciphertext $\mathbf{c} = (\mathbf{u}, \mathbf{v}) \in R^2$ under public key $(\mathbf{h}, \mathbf{s}) \in R^2$. On obtaining M ciphertexts $\mathbf{c}^{(i)} = (\mathbf{u}^{(i)}, \mathbf{v}^{(i)})$, $1 \leq i \leq M$, we derive M target syndromes. Decoding a single of these M instances recovers the corresponding $m^{(i)}$ from $\mathbf{v}^{(i)}$, and therefore the corresponding session key $K^{(i)}$ via a KDF application.

For HQC-1 our attack can recover 1-out-of- M session keys within bit complexity less than 143 for $M \geq 2^{34}$ instances.

Common Code HQC: 1-out-of- M Secret Key Recovery. An HQC public key is a tuple $(\mathbf{h}, \mathbf{s}) \in R^2$ where $\mathbf{s} = \mathbf{x} + \mathbf{h} \cdot \mathbf{y}$ for some small weight secret key $(\mathbf{x}, \mathbf{y})$. We consider a *Common Code setting*, where all M users share the same code $\mathbf{h}$ as a system-wide parameter. Per se, it is unclear whether a common $\mathbf{h}$ reduces security.

We now obtain a 1-out-of- M Common Code HQC *secret key recovery* attack with nM syndromes that recovers a single secret key $(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})$. Interestingly, recovery of 1-out-of- M HQC-1 secret keys in the Common Code setting takes bit complexity below 143 using only $M \geq 2^{10}$ public keys. Thus, one should clearly avoid the usage of Common Code HQC.

BIKE: 1-out-of- M Session Key Recovery. Similar to HQC, BIKE uses a public key $\mathbf{h} \in R$ and encrypts a random message m as $\mathbf{c} = (\mathbf{u}, v)$ where $\mathbf{u} = \mathbf{e}_0 + \mathbf{h} \cdot \mathbf{e}_1$. From M instances we derive nM syndromes. Solving this DOOM instance yields some $(\mathbf{e}_0^{(i)}, \mathbf{e}_1^{(i)})$. This immediately reveals $m^{(i)}$ via $v^{(i)}$, and therefore the session key $K^{(i)}$ by a KDF application.

For all instantiations (BIKE-1, BIKE-3, BIKE-5), we drop below the targeted security levels in the multi-instance setting already using $M \geq 2^{11}$, $M \geq 2^{20}$ and $M \geq 2^{13}$ instances respectively. This shows that BIKE does not provide sufficiently conservative parameter choices against multi-instance attacks.

Classic McEliece: 1-out-of- M Session Key Recovery. Classic McEliece uses a small weight vector $\mathbf{e} \in \mathbb{F}_2^n$ for deriving a session key K . A McEliece public key is a parity check matrix $P = [I_{n-k} \mid T]$, and the KEM provides a ciphertext $\mathbf{c} = P\mathbf{e}$. On obtaining M encapsulations, we directly derive the M syndromes $\mathbf{c}^{(i)}$, $i \leq M$. Solving a DOOM instance with P and these syndromes yields a single $\mathbf{e}^{(i)}$, from which we derive the corresponding session key $K^{(i)}$ via a KDF application.

As a result, for `mceliece3488-64` we drop below the security level for $M \geq 2^{21}$ instances.

DOOM Algorithms for Dumer-Stern and MMT. We provide explicit DOOM algorithms based on Dumer-Stern [Dum89, Ste88] and MMT [MMT11] information set decoding in Appendix A. Especially, we provide a novel MMT-DOOM algorithm that is capable of balancing the cost among all lists, improving over [EMZ22].

The code we used to compute our numerical results is available at

<https://github.com/Gabsadio/Multi-Instance-KEM>.

Related Work. Glabush, Hövelmanns and Stebila [GHS26] analyse the multi-target security of certain KEMs, including HQC. More specifically, they define a

multi-target version of IND-CCA security and show that the multi-target IND-CCA security of KEMs using a salted variant of the Fujisaki-Okamoto transform [FO13] reduces tightly to the multi-target IND-CPA security of the underlying public-key encryption (PKE). In this work, we analyse the multi-target IND-CPA security via 1-out-of- M attacks for HQC, BIKE and Classic McEliece.

For LWE-based schemes, Bernstein [Ber22] shows that 1-out-of- M attacks also lead to asymptotic speedups. His analysis is based on standard lattice heuristics, but the achievable speedups are hard to quantify for concrete lattice schemes. Bernstein’s as well as our result question the common cryptographic practice of focussing solely on single-instance security for a proper parameter choice.

This practice seemed to be justified for pre-quantum problems like discrete logarithms. Let T be the time required for solving a single dlog problem. Although it is known that solving M discrete logarithms can all be solved in time $\sqrt{MT}$ (instead of the trivial MT), solving 1-out-of- M discrete logarithms still costs time T [KS01,FJM14,CK18,MO25]. Thus, the discrete logarithm problem does not degrade as a function of M , as opposed to LWE- and code-based problems.

Organisation. In Section 2, we describe the DOOM technique. Details on how to instantiate DOOM with Dumer-Stern and MMT are provided in Appendix A. Section 3 provides our 1-out-of- M HQC session key recovery attack, and our Common Code HQC secret key recovery attack. Our 1-out-of- M session key recovery attacks for BIKE and McEliece are given in Sections 4 and 5.

2 Decoding One Out of Many (DOOM)

Notation. For $e \in \mathbb{F}_2^n$ let $w(e)$ denote its Hamming weight, i.e. the number of non-zero entries. We define $\mathbb{F}_2^n(w)$ to be the set of vectors $e \in \mathbb{F}_2^n$ with $w(e) = w$.

When an element x is sampled uniformly at random from a set S , we write $x \leftarrow_{\$} S$. The action of a permutation $\pi \in \mathfrak{S}_n$ on a matrix P with n columns is denoted as P^π . With P^T we denote the transpose of P .

Definition 1 (1-out-of- M decoding (DOOM)). Let $P \in \mathbb{F}_2^{(n-k) \times n}$ be the parity check matrix of a linear $[n, k]$ -code, and $w \in \mathbb{N}$. Let $s^{(i)} \in \mathbb{F}_2^{n-k}$ for $i \in \{1, \dots, M\}$, be syndromes such that there exist $e^{(i)} \in \mathbb{F}_2^n(w)$ satisfying $Pe^{(i)} = s^{(i)}$ for all i . In the 1-out-of- M Decoding problem (DOOM) one has to find on input $P, w, s^{(1)}, \dots, s^{(M)}$ a single $e^{(i)} \in \mathbb{F}_2^n(w)$ with $Pe^{(i)} = s^{(i)}$.

Parity-Bit Trick. As suggested in [EMZ22], since $w(e^{(i)}) = \langle \mathbf{1}^n, e^{(i)} \rangle = w \bmod 2$ we append for our analysis the vector $\mathbf{1}^n$ to P , and the parity $w \bmod 2$ to $s^{(i)}$, i.e. we use that

$$\begin{bmatrix} P \\ \mathbf{1} \dots \mathbf{1} \end{bmatrix} \cdot e^{(i)} = \begin{bmatrix} s^{(i)} \\ w \bmod 2 \end{bmatrix}.$$

Generic DOOM Algorithm. For solving the DOOM problem from Definition 1 we use information set decoding (ISD) algorithms based on Dumer-Stern (DS) [Dum89,Ste88] and MMT [MMT11]. In a nutshell, these algorithms act as follows.

We start by randomly permuting the columns of the parity-check matrix P . The permuted matrix is transformed into semi-systematic form by performing a partial Gaussian elimination on the rows of P^{-1} and on all syndromes $\mathbf{s}^{(1)}, \dots, \mathbf{s}^{(M)}$, see also Figure 1.

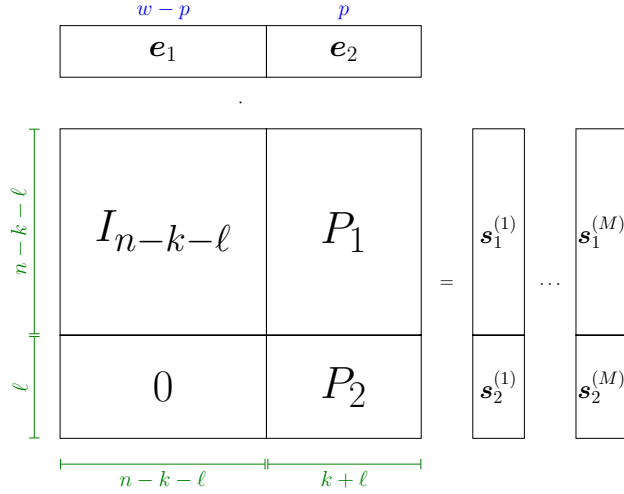


Fig. 1: Semi-systematic form after partial Gaussian elimination

For each column permutation we enumerate vectors $\mathbf{e}_2 \in \mathbb{F}_2^{k+\ell}(p)$. Thus, our DOOM algorithm is successful, if there exists a solution $\mathbf{e}^{(i)}$ from the DOOM problem (Definition 1) such that the column permutation leaves weight p on the last $k + \ell$ coordinates of $\mathbf{e}^{(i)}$. We summarize our complete algorithm **Generic DOOM** in Algorithm 1.

Dumer-Stern and MMT only differ in the way how the enumeration of $\mathbf{e}_2 \in \mathbb{F}_2^{k+\ell}(p)$ is performed. The technical details of Dumer-Stern and MMT are not important to understand our 1-out-of- M attacks on code-based KEMs. For completeness, we provide the details, including a full bit complexity runtime analysis, in Appendix A.

¹ Possible with probability at least $\frac{1}{4}$.

Algorithm 1: Generic-DOOM

Input: Parity check $P \in \mathbb{F}_2^{(n-k) \times n}$; weight w ; syndromes $\mathbf{s}^{(1)}, \dots, \mathbf{s}^{(M)} \in \mathbb{F}_2^{n-k}$
Output: $\mathbf{e} \in \mathbb{F}_2^n(w)$ with $P\mathbf{e} = \mathbf{s}^{(i)}$ for one $i \in \{1, \dots, M\}$
Parameters : $0 \leq p \leq w$, $\max(0, p-k) \leq \ell \leq n-k-w+p$

repeat
 repeat
 | Permute columns of P according to permutation $\pi \leftarrow_{\$} \mathfrak{S}_n$;
 until *first $n-k-\ell$ rows and columns of P^π have full rank*;
 $(G, P_1, P_2) \leftarrow \text{PartialGaussianElimination}(P^\pi, n-k-\ell)$;
 $(\mathbf{s}_1^{(i)}, \mathbf{s}_2^{(i)})^T := G\mathbf{s}^{(i)}$ for all $i \in \{1, \dots, M\}$;
 Enumerate $\mathbf{e}_2 \in \mathbb{F}_2^{k+\ell}(p)$, $i \in \{1, \dots, M\}$ with $\mathbf{s}_2^{(i)} = P_2\mathbf{e}_2$;
 $\mathbf{e}_1 := \mathbf{s}_1^{(i)} - P_1\mathbf{e}_2$;
 if $w(\mathbf{e}_1) = w-p$ **then return** $i, ((\mathbf{e}_1, \mathbf{e}_2)^\pi)^T$;

3 Hamming Quasi-Cyclic (HQC)

Let $R := \mathbb{F}_2[X]/(X^n - 1)$. In this section we describe our 1-out-of- M session key recovery algorithm for the same public key $(\mathbf{h}, \mathbf{s}) \in R^2$. Moreover, we describe our 1-out-of- M secret key recovery in a *Common Code* setting, where all users share the same $\mathbf{h}$.

Table 1: HQC parameters

	HQC-1	HQC-3	HQC-5
λ	128	192	256
$(n, \bar{\ell})$	(17669, 5)	(35851, 11)	(57637, 37)
w_e	75	114	149
w	66	100	131

3.1 HQC 1-out-of- M Session Key Recovery

HQC uses a public concatenated Reed-Muller and Reed-Solomon code

$$C' : \mathbb{F}_2^\lambda \rightarrow \mathbb{F}_2^{n-\bar{\ell}},$$

with an efficient decoding routine. See Table 1 for values of λ , n , $\bar{\ell}$. Let $G' \in \mathbb{F}_2^{(n-\bar{\ell}) \times \lambda}$ be a generator matrix of C' , i.e. $C' = \{G' \cdot m \mid m \in \mathbb{F}_2^\lambda\}$.

The HQC encapsulation routine is provided in Algorithm 2. For ease of exposition we leave out technical details that are irrelevant for our attack.

Algorithm 2: HQC.Encaps (simplified)

Input: public key $(\mathbf{h}, \mathbf{s}) \in R^2$
Output: session key $K \in \{0, 1\}^{256}$; encapsulation
 $c = (\mathbf{u}, \mathbf{v}, \text{salt}) \in R \times \mathbb{F}_2^{n-\bar{\ell}} \times \{0, 1\}^{128}$
 $m \leftarrow_{\$} \mathbb{F}_2^\lambda$; $\text{salt} \leftarrow_{\$} \{0, 1\}^{128}$; $\mathbf{r}_1, \mathbf{r}_2, \mathbf{e} \leftarrow_{\$} \mathbb{F}_2^n(w_e)$;
 $\mathbf{u} := \mathbf{r}_1 + \mathbf{h} \cdot \mathbf{r}_2$;
 $\mathbf{v} := G' \cdot m + \text{Truncate}_{\bar{\ell}}(\mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e})$;
 $K := \text{KDF}(pk, m, \text{salt})$;
return $K, c := (\mathbf{u}, \mathbf{v}, \text{salt})$

Idea of the attack. The goal of our attack is recovery of m from $(\mathbf{u}, \mathbf{v})$. Recall that

$$\begin{aligned} \mathbf{u} &:= \mathbf{r}_1 + \mathbf{h} \cdot \mathbf{r}_2 \\ \mathbf{v} &:= G' \cdot m + \text{Truncate}_{\bar{\ell}}(\mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e}), \end{aligned} \quad (2)$$

where $\text{Truncate}_{\bar{\ell}}: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^{n-\bar{\ell}}$ simply removes the last $\bar{\ell}$ coordinates. This truncation can be realized by multiplying from the left with

$$T_{\bar{\ell}} := \begin{bmatrix} I_{n-\bar{\ell}} & \mathbf{0}^{(n-\bar{\ell}) \times \bar{\ell}} \end{bmatrix} \in \mathbb{F}_2^{(n-\bar{\ell}) \times n}. \quad (3)$$

Since $G' \in \mathbb{F}_2^{(n-\bar{\ell}) \times \lambda}$ of C' is known, we can efficiently compute a parity check matrix

$$P' \in \mathbb{F}_2^{(n-\bar{\ell}-\lambda) \times (n-\bar{\ell})} \quad \text{such that } P' \cdot G' = \mathbf{0} \in \mathbb{F}_2^{(n-\bar{\ell}-\lambda) \times \lambda}. \quad (4)$$

Thus, we can rewrite Eq.(2) in matrix notation as

$$\begin{aligned} \mathbf{u} &= \mathbf{r}_1 + \text{rot}(\mathbf{h}) \cdot \mathbf{r}_2 \\ P' \cdot \mathbf{v} &= P' \cdot T_{\bar{\ell}} \cdot \text{rot}(\mathbf{s}) \cdot \mathbf{r}_2 + P' \cdot T_{\bar{\ell}} \cdot \mathbf{e}, \end{aligned} \quad (5)$$

thereby eliminating m .

Our attack now recovers the remaining unknowns $\mathbf{r}_1, \mathbf{r}_2, \mathbf{e}$ from Eq.(5). Therefore, we can remove the term $\mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e}$ from $\mathbf{v}$ in Eq.(2), allowing us to recover m and to recompute the session key K .

HQC Session Key Attack. We gather M encapsulations

$$\mathbf{c}^{(i)} = (\mathbf{u}^{(i)}, \mathbf{v}^{(i)}, \text{salt}^{(i)}) \quad \text{for } i \in \{1, \dots, M\},$$

of corresponding session keys $K^{(i)}$. Let us define

$$P := \begin{bmatrix} I_n & \text{rot}(\mathbf{h}) & \mathbf{0}^{n \times n} \\ \mathbf{0}^{(n-\bar{\ell}-\lambda) \times n} & P' \cdot T_{\bar{\ell}} \cdot \text{rot}(\mathbf{s}) & P' \cdot T_{\bar{\ell}} \\ 1 \dots 1 & 0 \dots 0 & 0 \dots 0 \\ 0 \dots 0 & 1 \dots 1 & 0 \dots 0 \\ 0 \dots 0 & 0 \dots 0 & 1 \dots 1 \end{bmatrix} \in \mathbb{F}_2^{(2n-\bar{\ell}-\lambda+3) \times 3n}.$$

Furthermore, we define M syndromes

$$\sigma_i := (\mathbf{u}^{(i)}, P' \cdot \mathbf{v}^{(i)}, w_e \bmod 2, w_e \bmod 2, w_e \bmod 2) \quad \text{with } i \in \{1, \dots, M\},$$

that correspond to the solutions $(\mathbf{r}_1^{(i)}, \mathbf{r}_2^{(i)}, \mathbf{e}^{(i)})$. See Table 1 for values of the error weight w_e . Unfortunately, the multiplication with $P' \cdot T_{\bar{\ell}}$ prevents us from obtaining more syndromes through the quasi-cyclic structure of the ring. We obtain a DOOM instance (Definition 1) as

$$(P, 3w_e, (\sigma_i)_{1 \leq i \leq M}).$$

An application of **Generic-DOOM** (Algorithm 1) then yields one solution $(\mathbf{r}_1^{(i)}, \mathbf{r}_2^{(i)}, \mathbf{e}^{(i)})$ together with i . Note that for HQC and all other KEMs examined in this paper, the parameters are chosen such that in expectation each syndrome decoding equation has a unique solution. Thus, our attack will recover the correct errors. Using Eq.(2), we obtain

$$\mathbf{v}^{(i)} + \text{Truncate}_{\bar{\ell}}(\mathbf{s} \cdot \mathbf{r}_2^{(i)} + \mathbf{e}^{(i)}) = \text{Encode}(m^{(i)}).$$

Since no error is left, we can recover $m^{(i)}$ as

$$m^{(i)} = \text{Decode}(\mathbf{v}^{(i)} + \text{Truncate}_{\bar{\ell}}(\mathbf{s} \cdot \mathbf{r}_2^{(i)} + \mathbf{e}^{(i)})).$$

This eventually yields the session key $K^{(i)} = \text{KDF}(pk, m^{(i)}, \text{salt}^{(i)})$. Our attack is summarised in Algorithm 3.

Algorithm 3: 1-out-of-M-HQC-Session-Keys

Input: public key $(\mathbf{h}, \mathbf{s}) \in R^2$; encapsulations $\mathbf{c}^{(1)}, \dots, \mathbf{c}^{(M)}$ with $\mathbf{c}^{(i)} = (\mathbf{u}^{(i)}, \mathbf{v}^{(i)}, \text{salt}^{(i)})$; error weight w_e

Output: session key $K^{(i)}$ for one $i \in \{1, \dots, M\}$

// see Eqs.(3), (4) for definition of $T_{\bar{\ell}}$ and P'

$$P := \begin{bmatrix} I_n & \text{rot}(\mathbf{h}) & \mathbf{0}^{n \times n} \\ \mathbf{0}^{(n-\bar{\ell}-\lambda) \times n} & P' \cdot T_{\bar{\ell}} \cdot \text{rot}(\mathbf{s}) & P' \cdot T_{\bar{\ell}} \\ 1 \dots 1 & 0 \dots 0 & 0 \dots 0 \\ 0 \dots 0 & 1 \dots 1 & 0 \dots 0 \\ 0 \dots 0 & 0 \dots 0 & 1 \dots 1 \end{bmatrix} \in \mathbb{F}_2^{(2n-\bar{\ell}-\lambda+3) \times 3n};$$

for $i = 1$ **to** M **do**

$\sigma_i := (\mathbf{u}^{(i)}, P' \cdot \mathbf{v}^{(i)}, w_e \bmod 2, w_e \bmod 2, w_e \bmod 2) \in \mathbb{F}_2^{2n-\bar{\ell}-\lambda+3}$;

end

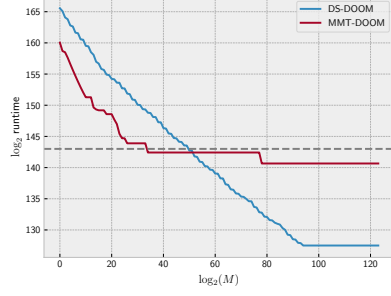
$(i, (\mathbf{r}_1^{(i)}, \mathbf{r}_2^{(i)}, \mathbf{e}^{(i)})) \leftarrow \text{Generic Doom}(P, 3w_e, (\sigma_i)_{1 \leq i \leq M})$;

$m^{(i)} := \text{Decode}(\mathbf{v}^{(i)} + \text{Truncate}_{\bar{\ell}}(\mathbf{s} \cdot \mathbf{r}_2^{(i)} + \mathbf{e}^{(i)}))$;

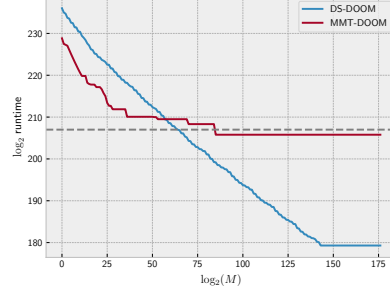
$K^{(i)} := \text{KDF}(pk, m^{(i)}, \text{salt}^{(i)})$;

return $K^{(i)}$

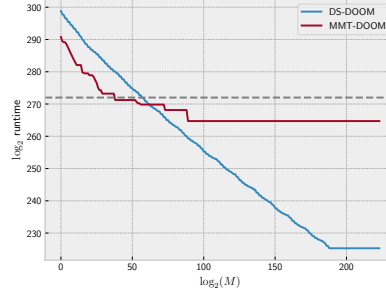
Results. For all HQC parameter sets from Table 1 we computed the bit complexities of both DOOM algorithms, Dumer-Stern (DS) and MMT. See Figure 2 for the resulting plots of the expected runtimes.



(a) HQC-1



(b) HQC-3



(c) HQC-5

Fig. 2: $\log_2$ runtimes of 1-out-of- M -HQC-Session-Keys

DS-DOOM. Let us first discuss the results for DS. We nicely observe the expected security degradation with the number M of instances. For all parameter sets, the (average) slope of degradation is (slightly below) $\frac{1}{2}$, as expected from a $\sqrt{M}$ speedup.

Notice that any 1-out-of- M attack with a speedup of $\sqrt{M}$ for single instance runtime T_1 runs in time $\max\{M, \frac{T_1}{\sqrt{M}}\}$. Thus, in Figure 2 we choose $M \leq T_1^{\frac{2}{3}}$, respectively $\log_2(M) \leq \frac{2}{3} \log_2(T_1)$. The *best runtime* is achieved for the maximal choice of M .

MMT-DOOM. Running **Generic-DOOM** with MMT yields improved runtimes for $M \leq 2^{52}$. Moreover, for technical reasons ² MMT cannot benefit from large M and thus it plateaus earlier than DS-DOOM.

Notice that for sufficiently large M , we fall short of the desired NIST security levels. This happens for **HQC-1** with $M \geq 2^{34}$ (MMT-DOOM), for **HQC-3** with $M \geq 2^{64}$ (DS-DOOM), and for **HQC-5** with $M \geq 2^{38}$ (MMT-DOOM).

Table 2 provides a summary of all bit complexities for time (T) and space (S) for the following three points of interest.

Single Instance: The single instance running time T_1 for $M = 1$ that is analysed in the HQC specification [GAMA⁺25].

Below Security Level: The point at which the expected runtime of **1-Out-of-M-HQC-Session-Keys** (Algorithm 3) falls below the NIST security levels of 143, 207 and 272 bits, respectively.

Best Runtime: The minimal runtime that we could achieve using a maximal amount of M .

Additionally, the last two columns of Table 2 provide the average and maximal $\log_M$ speedups.

Table 2: Summary of bit complexities of **1-out-of-M-HQC-Session-Keys**

		single instance		below security level			best runtime			$\log_M$ speedups	
		T_1	S_1	M	T	S	M	T	S	avg.	max.
HQC-1	DS	165.55	30.80	50	142.73	65.11	94	127.50	109.11	0.4834	0.7324
	MMT	160.04	53.65	34	142.42	63.87	78	140.66	131.38	0.4940	1.3438
HQC-3	DS	236.01	32.84	64	206.99	80.15	143	179.30	159.13	0.4716	1.0018
	MMT	228.84	57.83	85	205.79	157.62	85	205.79	157.62	0.4774	1.3494
HQC-5	DS	298.83	34.21	57	271.83	73.81	188	225.28	204.81	0.4581	1.0013
	MMT	290.78	60.64	38	271.23	71.98	89	264.71	166.56	0.4714	1.3536

3.2 Common Code HQC: 1-out-of- M Secret Key Recovery

Let us define a *Common Code* $\mathbf{h}$ for all users as a system-wide parameter.

1-out-of- M Common Code HQC Secret Key Attack. We collect M Common Code HQC public keys $\mathbf{s}^{(1)}, \dots, \mathbf{s}^{(m)}$ from Algorithm 4 satisfying

$$\mathbf{s}^{(i)} = \mathbf{x}^{(i)} + \mathbf{h} \cdot \mathbf{y}^{(i)} \quad \text{for } i = 1, \dots, M.$$

² The base lists in MMT-DOOM are too small to integrate huge M

Algorithm 4: Common Code HQC.KeyGeneration

Input: system-wide $\mathbf{h} \in R$
Output: Common Code HQC key pair (pk, sk)

$\mathbf{x}, \mathbf{y} \leftarrow_{\$} \mathbb{F}_2^n(w);$
 $\mathbf{s} := \mathbf{x} + \mathbf{h} \cdot \mathbf{y};$
 $pk := \mathbf{s}; sk := (\mathbf{x}, \mathbf{y});$
return (pk, sk)

Let us define the parity-check matrix

$$P := \begin{bmatrix} I_n & \text{rot}(\mathbf{h}) \\ 1 \dots 1 & 0 \dots 0 \\ 0 \dots 0 & 1 \dots 1 \end{bmatrix} \in \mathbb{F}_2^{(n+2) \times 2n}$$

and the nM syndromes

$$\mathbf{s}_{i,j} := (X^j \cdot \mathbf{s}^{(i)}, w \bmod 2, w \bmod 2) \in \mathbb{F}_2^{n+2} \quad \text{for } 1 \leq i \leq M, 0 \leq j < n.$$

This gives us a DOOM instance (Definition 1)

$$(P, 2w, (\mathbf{s}_{i,j})_{1 \leq i \leq M, 0 \leq j < n}).$$

An application of **Generic-DOOM** then yields a single secret key $sk^{(i)} = (\mathbf{x}^{(i)}, \mathbf{y}^{(i)})$ together with the user's id i , after removing X^j . Our attack is summarised in Algorithm 5.

Algorithm 5: 1-out-of-M-Common-Code-HQC-Secret-Keys

Input: common code $\mathbf{h}$; public keys $\mathbf{s}^{(1)}, \dots, \mathbf{s}^{(M)}$; weight w
Output: secret key $sk^{(i)} = (\mathbf{x}^{(i)}, \mathbf{y}^{(i)})$ for one $i \in \{1, \dots, M\}$

$P := \begin{bmatrix} I_n & \text{rot}(\mathbf{h}) \\ 1 \dots 1 & 0 \dots 0 \\ 0 \dots 0 & 1 \dots 1 \end{bmatrix} \in \mathbb{F}_2^{(n+2) \times 2n};$

for $i = 1$ **to** M **do**

for $j = 0$ **to** $n - 1$ **do**

$\mathbf{s}_{i,j} := (X^j \cdot \mathbf{s}^{(i)}, w \bmod 2, w \bmod 2) \in \mathbb{F}_2^{n+2};$

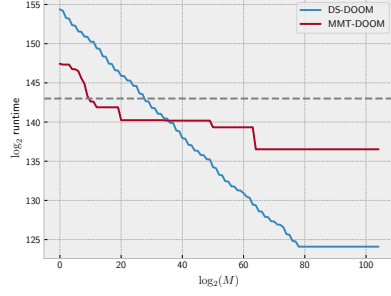
end

end

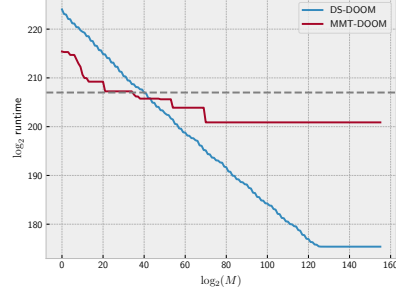
$((i, j), (X^j \cdot \mathbf{x}^{(i)}, X^j \cdot \mathbf{y}^{(i)})) \leftarrow \text{Generic-Doom}(P, 2w, (\mathbf{s}_{i,j})_{1 \leq i \leq M, 0 \leq j < n});$

$sk^{(i)} := (\mathbf{x}^{(i)}, \mathbf{y}^{(i)});$

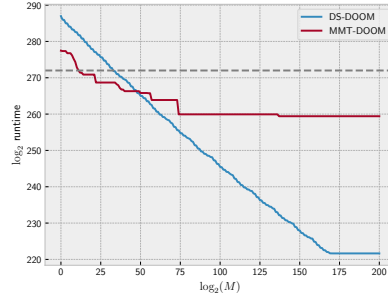
return $sk^{(i)}$



(a) HQC-1



(b) HQC-3



(c) HQC-5

Fig. 3: $\log_2$ runtimes of 1-out-of-M-Common-Code-HQC-Secret-Keys

Results. For all parameter sets of HQC from Table 1 we computed the expected runtimes of our DOOM variants of DS and MTT as a function of the number M of public keys, see Figure 3.

Our results are qualitatively similar to those for HQC session key recovery from Section 3.1, showing first a security degradation with slope roughly $\sqrt{M}$, and second an MMT improvement for small $M < 2^{36}$. However, notice that we quantitatively obtain significantly lower runtimes for secret key recovery than for session key recovery.

As a consequence we fall below the NIST security levels with MMT-DOOM already for small values of M , namely for 2^{10} (HQC-1), 2^{35} (HQC-3), and 2^{12} (HQC-5) public keys, see also Table 3. Our results show that HQC should not be used in a Common Code setting.

Table 3: Bit complexities of 1-out-of-M-Common-Code-HQC-Secret-Keys

		single instance		below security level			best runtime			$\log_M$ speedups	
		T_1	S_1	M	T	S	M	T	S	avg.	max.
HQC-1	DS	154.37	29.80	28	142.67	56.22	78	124.10	106.23	0.4023	0.5428
	MMT	147.43	42.17	10	142.61	54.36	64	136.52	127.20	0.2259	0.4823
HQC-3	DS	224.04	31.84	40	206.95	70.26	127	175.40	156.37	0.4280	0.9478
	MMT	215.40	45.33	35	206.61	104.47	70	200.88	157.51	0.2522	0.4953
HQC-5	DS	286.96	33.21	33	271.83	64.63	170	221.65	200.94	0.4275	0.9576
	MMT	277.46	47.45	12	271.41	61.43	137	259.41	250.41	0.2230	0.5043

4 Bit-Flipping Key Encapsulation (BIKE)

In this section we describe our 1-out-of- M session key recovery algorithm for the same BIKE public key $\mathbf{h} \in R := \mathbb{F}_2^n[X]/(X^n - 1)$. The encapsulation algorithm of BIKE is provided in Algorithm 6, where we leave out technical details irrelevant for our attack.

Algorithm 6: BIKE.Encaps (simplified)

Input: public key $\mathbf{h} \in R$
Output: session key $K \in \{0, 1\}^{256}$; encapsulation $c \in R \times \{0, 1\}^{256}$

$m \leftarrow_{\$} \{0, 1\}^{256}$; $(\mathbf{e}_0, \mathbf{e}_1) := \text{Hash}_1(m, \mathbf{h})$;
 $\mathbf{u} := \mathbf{e}_0 + \mathbf{h} \cdot \mathbf{e}_1$;
 $v := m \oplus \text{Hash}_2(\mathbf{e}_0, \mathbf{e}_1)$;
 $c := (\mathbf{u}, v)$;
 $K := \text{KDF}(m, c)$;
return K, c

BIKE.Encaps (Algorithm 7) uses a hash function

$$\text{Hash}_1 : \{0, 1\}^{256} \times R \rightarrow \{(\mathbf{e}_0, \mathbf{e}_1) \in R \times R \mid w(\mathbf{e}_0, \mathbf{e}_1) = w\}.$$

I.e. Hash_1 maps to two ring elements with total Hamming weight w . For values of w see Table 4.

Table 4: BIKE parameters

	BIKE-1	BIKE-3	BIKE-5
n	12323	24659	40973
w	134	199	264

Idea of our attack. As in the HQC Session Key Recovery Attack (Section 3.1), our goal is to recover the message m . Consider

$$v := m \oplus \text{Hash}_2(\mathbf{e}_0, \mathbf{e}_1). \quad (6)$$

If we recover $\mathbf{e}_0, \mathbf{e}_1$ from $\mathbf{u} = \mathbf{e}_0 + \mathbf{h} \cdot \mathbf{e}_1$, then we can extract m from v and compute the session key K .

4.1 1-out-of-M BIKE Session Key Recovery.

We gather M encapsulations

$$c^{(i)} = (\mathbf{u}^{(i)}, v^{(i)}) \quad \text{for } i \in \{1, \dots, M\},$$

generated with the same public key $\mathbf{h}$. Let us define the parity-check matrix

$$P := \begin{bmatrix} I_n & \text{rot}(\mathbf{h}) \\ 1 \dots 1 & 1 \dots 1 \end{bmatrix} \in \mathbb{F}_2^{(n+1) \times 2n}.$$

Moreover, we define nM many syndromes

$$\mathbf{u}_{i,j} := (X^j \cdot \mathbf{u}^{(i)}, w \bmod 2) \in \mathbb{F}_2^{n+1} \quad \text{for } 1 \leq i \leq M, 0 \leq j < n$$

that correspond to the solutions $(X^j \cdot \mathbf{e}_0, X^j \cdot \mathbf{e}_1)$ with $w((X^j \cdot \mathbf{e}_0, X^j \cdot \mathbf{e}_1)) = w$. Hence, we obtain the following DOOM instance (Definition 1):

$$(P, w, (\mathbf{u}_{i,j})_{1 \leq i \leq M, 0 \leq j < n}).$$

An application of **Generic-DOOM** (Algorithm 1) yields a solution $(X^j \cdot \mathbf{e}_0^{(i)}, X^j \cdot \mathbf{e}_1^{(i)})$ together with i, j . We remove X^j to get $(\mathbf{e}_0^{(i)}, \mathbf{e}_1^{(i)})$. Using Eq.(6), we recover

$$m^{(i)} = v^{(i)} \oplus \text{Hash}_2(\mathbf{e}_0^{(i)}, \mathbf{e}_1^{(i)}).$$

Finally, we compute the corresponding session key as

$$K^{(i)} = \text{KDF}(m^{(i)}, c^{(i)}).$$

Our attack is summarised in Algorithm 7.

Algorithm 7: 1-out-of-M-BIKE-Session-Keys

Input: public key $\mathbf{h} \in R$; encapsulations $c^{(1)}, \dots, c^{(M)}$ with $c^{(i)} = (\mathbf{u}^{(i)}, v^{(i)}) \in R \times \{0, 1\}^{256}$; error weight w

Output: session key $K^{(i)}$ for one $i \in \{1, \dots, M\}$

$P := \begin{bmatrix} I_n & \text{rot}(\mathbf{h}) \\ 1 \dots 1 & 1 \dots 1 \end{bmatrix} \in \mathbb{F}_2^{(n+1) \times 2n}$;

for $i = 1$ **to** M **do**

for $j = 0$ **to** $n - 1$ **do**

$\mathbf{u}_{i,j} := (X^j \cdot \mathbf{u}^{(i)}, w \bmod 2) \in \mathbb{F}_2^{n+1}$

end

end

$((i, j), (X^j \cdot \mathbf{e}_0^{(i)}, X^j \cdot \mathbf{e}_1^{(i)})) \leftarrow \text{Generic-DOOM}(P, w, (\mathbf{u}_{i,j})_{1 \leq i \leq M, 0 \leq j < n})$;

$m^{(i)} := v^{(i)} \oplus \text{Hash}_2(\mathbf{e}_0^{(i)}, \mathbf{e}_1^{(i)})$;

$K^{(i)} := \text{KDF}(m^{(i)}, c^{(i)})$;

return $K^{(i)}$

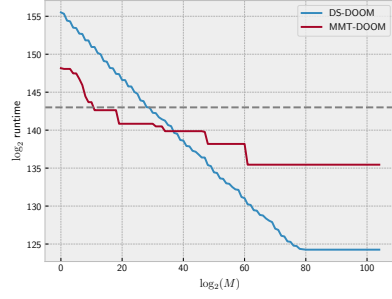
Results. For all proposed BIKE parameter sets (Table 4) we computed the bit complexities of DS-DOOM and MMT-DOOM as a function of the number M of encapsulations. See Figure 4 for the resulting plots of the runtimes as a function of M .

As in Section 3, we observe a security degradation with slope roughly $\sqrt{M}$, and MMT performing better than DS for small M . We only need 2^{11} , 2^{20} respectively 2^{13} encapsulations to get below the targeted security levels with MMT-DOOM, see also Table 5.

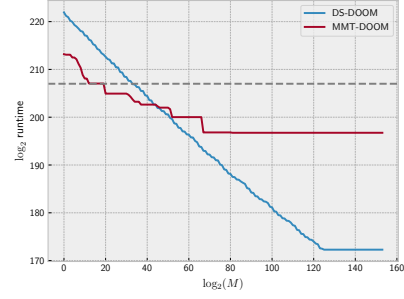
Table 5: Bit complexities of 1-out-of-M-BIKE-Session-Keys

		single instance below security level					best time			$\log_M$ speedups	
		T_1	S_1	M	T	S	M	T	S	avg.	max.
BIKE-1	DS	155.51	28.76	29	142.97	56.18	80	124.26	107.10	0.4200	0.5401
	MMT	148.17	40.54	11	142.63	52.47	61	135.45	126.46	0.2540	0.5033
BIKE-3	DS	221.93	32.24	33	206.90	62.18	125	172.29	154.19	0.4356	0.7214
	MMT	213.18	43.64	20	204.94	66.09	81	196.74	176.28	0.2693	0.5115
BIKE-5	DS	288.20	32.23	35	271.63	65.65	171	219.73	201.65	0.4433	0.9554
	MMT	278.26	45.93	13	271.69	59.57	132	255.51	246.60	0.2499	0.5199

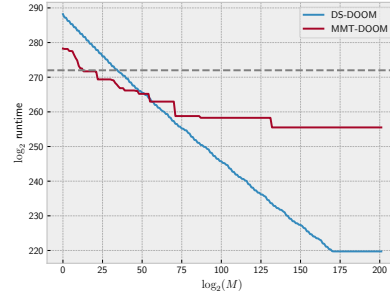
We conclude that BIKE’s public keys should not be used long-term. The BIKE team already mentions in [ABB⁺24, section 3.5] that “BIKE’s design fits well with ephemeral keys”. Our results quantify BIKE’s security degradation for public key reuse.



(a) BIKE-1



(b) BIKE-3



(c) BIKE-5

Fig. 4: $\log_2$ runtimes of 1-out-of-M-BIKE-Session-Keys

5 Classic McEliece

The encapsulation routine of Classic McEliece [BCC⁺22] is given in Algorithm 8. See Table 6 for the proposed parameter sets.

Table 6: Classic McEliece parameters

	3488-64	4608-96	6688-128	6960-119	8192-128
n	3488	4608	6688	6960	8192
k	2720	3360	5024	5413	6528
w	64	96	128	119	128

Algorithm 8: ClassicMcEliece.Encaps (simplified)

Input: public key $T \in \mathbb{F}_2^{(n-k) \times k}$
Output: session key $K \in \{0, 1\}^{256}$; encapsulation $\mathbf{c} \in \mathbb{F}_2^{n-k}$
 $\mathbf{e} \leftarrow_{\$} \mathbb{F}_2^n(w)$;
 $\mathbf{c} := [I_{n-k} \mid T] \cdot \mathbf{e}$;
 $K := \text{KDF}(\mathbf{e}, \mathbf{c})$;
return $K, \mathbf{c}$

5.1 1-out-of-M McEliece Session Key Recovery.

We collect M syndromes $\mathbf{c}^{(1)}, \dots, \mathbf{c}^{(M)}$. Let us define the parity check matrix

$$P := \begin{bmatrix} I_{n-k} & T \\ 1 \dots 1 & 1 \dots 1 \end{bmatrix} \in \mathbb{F}_2^{(n-k+1) \times n}.$$

Moreover, we define the M syndromes

$$\mathbf{s}^{(i)} = (\mathbf{c}^{(i)}, w \bmod 2) \in \mathbb{F}_2^{n-k+1} \quad \text{for } 1 \leq i \leq M,$$

corresponding to the solutions $\mathbf{e}^{(i)} \in \mathbb{F}_2^n(w)$.

Thus, we obtain a DOOM instance (Definition 1)

$$(P, w, \mathbf{s}^{(1)}, \dots, \mathbf{s}^{(M)}).$$

An application of **Generic-Doom** (Algorithm 1) yields a solution $\mathbf{e}^{(i)}$, together with i . We then derive the session key

$$K^{(i)} = \text{KDF}(\mathbf{e}^{(i)}, \mathbf{c}^{(i)}).$$

Our attack is summarised in Algorithm 9.

Algorithm 9: 1-out-of-M-McEliece-Session-Keys

Input: public key $T \in \mathbb{F}_2^{(n-k) \times k}$; encapsulations $\mathbf{c}^{(1)}, \dots, \mathbf{c}^{(M)}$; weight w
Output: session key $K^{(i)}$ for one $i \in \{1, \dots, M\}$
 $P := \begin{bmatrix} I_{n-k} & T \\ 1 \dots 1 & 1 \dots 1 \end{bmatrix} \in \mathbb{F}_2^{(n-k+1) \times n}$;
for $i = 1$ **to** M **do**
 $\mathbf{s}^{(i)} := (\mathbf{c}^{(i)}, w \bmod 2) \in \mathbb{F}_2^{n-k+1}$;
end
 $(i, \mathbf{e}^{(i)}) \leftarrow \text{Generic-DOOM}(P, w, \mathbf{s}^{(1)}, \dots, \mathbf{s}^{(M)})$;
 $K^{(i)} := \text{KDF}(\mathbf{e}^{(i)}, \mathbf{c}^{(i)})$;
return $K^{(i)}$

Results. For all parameter sets (Table 6) we computed the expected performances of both DOOM algorithms, DS and MMT, as a function of the number M of instances (see Figure 5).

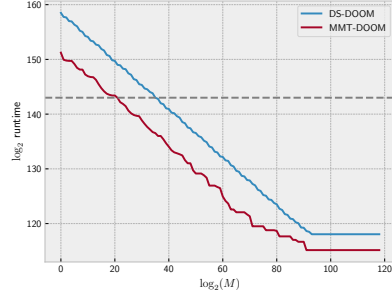
For parameter sets `mceliece3488-64` (NIST level 1) and `mceliece8192-128` (NIST level 5) we fall below the desired security levels for $M \geq 2^{21}$, respectively $M \geq 2^{59}$ (see also Table 7).

Table 7: Bit complexities of 1-out-of-M-McEliece-Session-Keys.

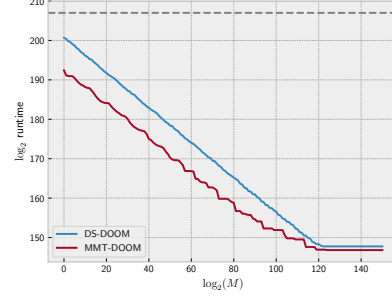
		single instance below security level					best time			$\log_M$ speedups	
		T_1	S_1	M	T	S	M	T	S	avg.	max.
3488-64	DS	158.50	33.59	35	143.00	67.72	93	118.05	102.51	0.4437	0.7445
	MMT	151.22	70.87	21	142.97	74.87	91	115.15	109.19	0.4339	1.2832
4608-96	DS	200.66	52.34	0	200.66	52.34	122	147.75	132.42	0.4390	0.4539
	MMT	192.43	73.31	0	192.43	73.31	124	146.81	139.42	0.4282	1.2806
6688-128	DS	275.72	89.79	9	271.66	81.53	174	199.24	184.85	0.4451	0.5956
	MMT	264.33	120.28	0	264.33	120.28	172	197.95	192.36	0.4309	1.2887
6960-119	DS	276.01	90.76	9	271.96	82.30	173	200.11	183.75	0.4409	0.4501
	MMT	264.30	134.86	0	264.30	134.86	170	197.96	191.84	0.4340	1.2917
8192-128	DS	311.02	110.09	88	271.54	150.24	198	223.43	208.86	0.4449	0.4627
	MMT	297.06	152.41	59	271.92	184.85	190	221.97	213.87	0.4328	1.2909

For all other parameters, our single instance MMT bit complexities are already below the desired security levels. Notice that to be in line with the methodology taken by the HQC [GAMA⁺25] and BIKE [ABB⁺24] team, we are using throughout our work a pure time metric, not penalizing memory costs as in [BCC⁺22]. Therefore, our single instance complexities for HQC and BIKE from Sections 3 and 4 coincide with [GAMA⁺25, ABB⁺24], but not with [BCC⁺22]. For a consistent and easy comparison, we stick to a pure time metric also for Classic McEliece.

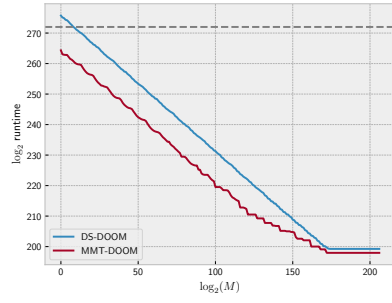
Independent of concrete complexities, Classic McEliece also shows the typical security degradation with factor roughly $\sqrt{M}$ in the multi-instance setting.



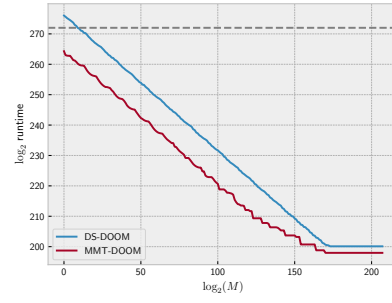
(a) mceliece3488-64



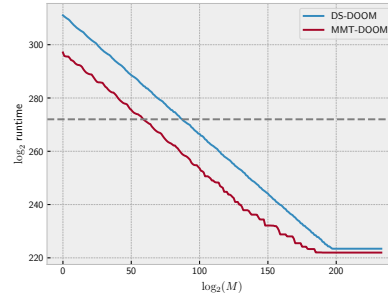
(b) mceliece4608-96



(c) mceliece6688-128



(d) mceliece6960-119



(e) mceliece8192-128

Fig. 5: $\log_2$ runtimes of 1-out-of-M-McEliece-Session-Keys

Acknowledgments. We are grateful to Ray A. Perlner for pointing out an improvement to our HQC 1-out-of- M Session Key Recovery in Section 3.1.

Alexander May was funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany’s Excellence Strategy - EXC 2092 CASA - 390781972.

References

- ABB⁺24. Nicolas Aragon, Paulo Barreto, Slim Bettaieb, et al. Bike: Bit flipping key encapsulation. https://bikesuite.org/files/v5.2/BIKE_Spec.2024.10.1.pdf, 2024.
- BCC⁺22. Daniel J. Bernstein, Tung Chou, Carlos Cid, et al. Classic mceliece: conservative code-based cryptography: cryptosystem specification. <https://classic.mceliece.org/nist.html>, 2022.
- Ber22. Daniel J. Bernstein. Multi-ciphertext security degradation for lattices. Cryptology ePrint Archive, Report 2022/1580, 2022.
- CK18. Henry Corrigan-Gibbs and Dmitry Kogan. The discrete-logarithm problem with preprocessing. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 415–447. Springer, Cham, April / May 2018.
- Dum89. Ilya Dumer. Two decoding algorithms for linear codes. *Problems of Information Transmission*, 25, 03 1989.
- EMZ22. Andre Esser, Alexander May, and Floyd Zveydinger. McEliece needs a break - solving McEliece-1284 and quasi-cyclic-2918 with modern ISD. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 433–457. Springer, Cham, May / June 2022.
- FJM14. Pierre-Alain Fouque, Antoine Joux, and Chrysanthi Mavromati. Multi-user collisions: Applications to discrete logarithm, Even-Mansour and PRINCE. In Palash Sarkar and Tetsu Iwata, editors, *ASIACRYPT 2014, Part I*, volume 8873 of *LNCS*, pages 420–438. Springer, Berlin, Heidelberg, December 2014.
- FO13. Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. *Journal of Cryptology*, 26(1):80–101, January 2013.
- GAMA⁺25. Philippe Gaborit, Carlos Aguilar-Melchor, Nicolas Aragon, et al. Hamming quasi-cyclic (hqc). https://pqc-hqc.org/doc/hqc_specifications_2025_08_22.pdf, 2025.
- GHS26. Lewis Glabush, Kathrin Hövelmanns, and Douglas Stebila. On the multi-target security of post-quantum key encapsulation mechanisms. *IACR Communications in Cryptology*, 3(1), 2026.
- KS01. Fabian Kuhn and René Struik. Random walks revisited: Extensions of Pollard’s rho algorithm for computing multiple discrete logarithms. In Serge Vaudenay and Amr M. Youssef, editors, *SAC 2001*, volume 2259 of *LNCS*, pages 212–229. Springer, Berlin, Heidelberg, August 2001.

- MMT11. Alexander May, Alexander Meurer, and Enrico Thomae. Decoding random linear codes in $\tilde{O}(2^{0.054n})$. In Dong Hoon Lee and Xiaoyun Wang, editors, *ASIACRYPT 2011*, volume 7073 of *LNCS*, pages 107–124. Springer, Berlin, Heidelberg, December 2011.
- MO25. Alexander May and Massimo Ostuzzi. Multiple group action dlogs with(out) precomputation. In Tibor Jager and Jiaxin Pan, editors, *PKC 2025: 28th International Conference on Theory and Practice of Public Key Cryptography, Part III*, volume 15676 of *Lecture Notes in Computer Science*, pages 364–387, Røros, Norway, May 12–15, 2025. Springer, Cham, Switzerland.
- Mor06. Kent E. Morrison. Integer sequences and matrices over finite fields. *Journal of Integer Sequences*, 9, 2006. Article 06.2.1.
- Nat. National Institute of Standards and Technology. Pqc standardization process. <https://csrc.nist.gov/pqc-standardization>.
- Sen11. Nicolas Sendrier. Decoding one out of many. In Bo-Yin Yang, editor, *Post-Quantum Cryptography - 4th International Workshop, PQCrypto 2011*, pages 51–67. Springer, Berlin, Heidelberg, November / December 2011.
- Ste88. Jacques Stern. A method for finding codewords of small weight. In *Proceedings of the 3rd International Colloquium on Coding Theory and Applications*, page 106–113, Berlin, Heidelberg, 1988. Springer-Verlag.

A Details of Our DOOM Algorithms

Let $\epsilon_{p,\ell,M}$ denote the probability that a single iteration of **Generic-DOOM** (Algorithm 1) returns a solution $\mathbf{e}$. The expected runtime is then given by

$$T_{p,\ell,M} := \frac{1}{\epsilon_{p,\ell,M}} \cdot (T_{\text{Init}}(p, \ell, M) + T_{\text{Enum}}(p, \ell, M)),$$

where $T_{\text{Init}}(p, \ell, M)$ and $T_{\text{Enum}}(p, \ell, M)$ are the (expected) runtimes of the initial transformation and the enumeration respectively. To be more precise, when performing our calculations, we give vector additions a cost linear in their length and matrix-vector products a cost equal to the product of the dimensions.

A good metric for the performance of a DOOM algorithm is the $\log_M$ speed-up gained from using M instances:

$$\log_M \left(\frac{T_{p_1, \ell_1, 1}}{T_{p_M, \ell_M, M}} \right).$$

For memory, the cost is given by

$$\max\{S_{\text{Init}}(p, \ell, M), S_{\text{Enum}}(p, \ell, M)\},$$

where $S_{\text{Init}}(p, \ell, M)$ and $S_{\text{Enum}}(p, \ell, M)$ denote the memory needed for the initial transformation and enumeration, respectively. We give storing a vector a cost linear in its length.

Concrete Parameter Selection. All DOOM algorithms are parametrised and we optimise these parameters to get a minimal runtime.

Further, if the computed runtime with a certain amount of instances is higher than the computed runtime with fewer instances, then we ignore the one with more instances. Thus, as a function in the number of instances provided, the runtime will be monotonically decreasing.

A.1 Cost of Initial Transformation

Since the upcoming DOOM algorithms perform the same initial transformation, let us analyse its cost at this point.

The first step of randomly permuting the columns of the parity-check matrix can be done in time n .

For the partial Gaussian elimination, we can perform it simultaneously on the permuted matrix and the syndromes by viewing them as a single matrix:

$$\begin{pmatrix} I_{n-k-\ell} & P_1 & \mathbf{s}_1^{(1)} & \cdots & \mathbf{s}_1^{(M)} \\ 0 & P_2 & \mathbf{s}_2^{(1)} & \cdots & \mathbf{s}_2^{(M)} \end{pmatrix} = G \cdot (P^\pi \mathbf{s}^{(1)} \cdots \mathbf{s}^{(M)}).$$

We perform $n - k - \ell$ steps, and in each step we update an $(n - k) \times (n + M)$ matrix. Thus, the partial Gaussian elimination can be done in time $(n - k - \ell)(n - k)(n + M)$.

This initial transformation is successful if the submatrix formed by the first $n - k - \ell$ rows and columns of the permuted matrix has full rank. The probability for this can be bounded from below by 0.28, cf. [Mor06, section 1.2]. In the computations, we will therefore divide the runtime for this initial transformation by this bound to account for the expected number of repetitions.

Concerning memory, we just need to store the initial parity-check matrix and all syndromes, as all operations can be done in place. So in total, we have

$$T_{\text{Init}}(p, \ell, M) = \frac{1}{0.28} (n + (n - k - \ell)(n - k)(n + M)) \quad \text{and} \\ S_{\text{Init}}(p, \ell, M) = (n - k)(n + M).$$

A.2 DOOM With Dumer-Stern (DS)

In DS-IsD, potential sub-solutions $\mathbf{e}_2 \in \mathbb{F}_2^{k+\ell}(p)$ are enumerated via Meet-in-the-Middle, or rather a 2-list algorithm. Our DOOM variant is a generalisation of this. For this, we split the vector into two parts $\mathbf{e}_2 = (\mathbf{e}_{21} \mathbf{e}_{22})^T$, which allows to transform the syndrome decoding equation in the following way:

$$\begin{aligned} \mathbf{s}_2^{(i)} &= P_2 \mathbf{e}_2 = P_2(\mathbf{e}_{21} \mathbf{0})^T + P_2(\mathbf{0} \mathbf{e}_{22})^T \\ \iff P_2(\mathbf{e}_{21} \mathbf{0})^T &= \mathbf{s}_2^{(i)} - P_2(\mathbf{0} \mathbf{e}_{22})^T. \end{aligned} \tag{7}$$

See Figure 6 for how we do the splitting.

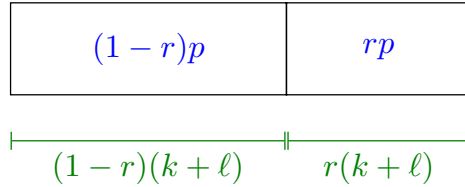


Fig. 6: Distribution of the length (green) and weight (blue) of $\mathbf{e}_2$ in DS-DOOM

Now two lists are filled with each side of Eq.(7), respectively:

$$\begin{aligned} L_1 &:= \left\{ P_2(\mathbf{e}_{21} \mathbf{0})^T : \mathbf{e}_{21} \in \mathbb{F}_2^{(1-r)(k+\ell)}((1-r)p) \right\} \\ L_2 &:= \left\{ \mathbf{s}_2^{(i)} - P_2(\mathbf{0} \mathbf{e}_{22})^T : i \in \{1, \dots, M\}, \mathbf{e}_{22} \in \mathbb{F}_2^{r(k+\ell)}(rp) \right\}. \end{aligned}$$

The parameter $r \in [0, 1/2]$ allows us to distribute the weight p such that the sizes of these initial lists are balanced, i.e. $|L_1| \approx |L_2|$.

We perform a matrix-vector multiplication for each element in both lists, and in L_2 we additionally perform a vector addition. Thus, the construction of these lists costs $(|L_1| + |L_2|) \cdot (k + \ell) \cdot \ell + |L_2| \cdot \ell$.

To get the list $L_1 \bowtie L_2$ containing the sub-solutions, we use a hash joining algorithm. For this, the smaller of the two lists is hashed into a hash map, while each element of the other list is computed on the fly and searched for in the hash map. This requires $\min\{|L_1|, |L_2|\} \cdot \ell$ bits of memory. In case of a match, this yields a sub-solution $\mathbf{e}_2 := (\mathbf{e}_{21} \mathbf{e}_{22})^T$ which can be checked directly whether it also yields a solution to the original problem, i.e. whether $w(\mathbf{s}_1^{(i)} - P_1 \mathbf{e}_2) = w - p$. This check involves a matrix-vector multiplication and we get $\mathbf{e}_2$ by adding the corresponding vectors from the initial lists, so in total the construction of $L_1 \bowtie L_2$ takes time $|L_1 \bowtie L_2| \cdot ((k + \ell) + (n - k - \ell) \cdot (k + \ell))$.

Under the assumption that P_2 behaves like a uniformly random linear map, the expected size of the joined list is given by

$$\mathbb{E}|L_1 \bowtie L_2| = \frac{|L_1| \cdot |L_2|}{2^\ell},$$

because the list elements are binary vectors of length ℓ that we fully match.

Expected Cost and Parameter Choice. The algorithm will be successful if at least one of the error vectors is permuted such that its weight w is distributed correctly. Here that means a weight of $w - p$ in the first $n - k - \ell$ non-zero entries, a weight of $(1 - r)p$ in the following $(1 - r)(k + \ell)$ entries and a weight of rp in the remaining $r(k + \ell)$ entries. For a fixed error vector, this happens with probability

$$\alpha_{p,\ell,r} := \frac{\binom{n-k-\ell}{w-p} \binom{(1-r)(k+\ell)}{(1-r)p} \binom{r(k+\ell)}{rp}}{\binom{n}{w}}.$$

Therefore, a permutation is good for at least one error vector with probability

$$\epsilon_{p,\ell,r,M} := 1 - (1 - \alpha_{p,\ell,r})^M = M\alpha_{p,\ell,r} + O((M\alpha_{p,\ell,r})^2).$$

Since $M\alpha_{p,\ell,r}$ is very small, we can safely approximate by

$$\epsilon_{p,\ell,r,M} \approx \min\{1, M\alpha_{p,\ell,r}\}.$$

The expected runtime of DS-DOOM is then given by

$$\begin{aligned} \frac{1}{\epsilon_{p,\ell,r,M}} \cdot \left(T_{\text{Init}}(p, \ell, M) + (|L_1| + |L_2|) \cdot (k + \ell) \cdot \ell + |L_2| \cdot \ell \right. \\ \left. + \frac{|L_1| \cdot |L_2|}{2^\ell} \cdot ((k + \ell) + (n - k - \ell) \cdot (k + \ell)) \right). \end{aligned} \quad (8)$$

For given p , choosing r such that $|L_1| \approx |L_2|$ is only possible if $M \leq \binom{k+\ell}{p}$ (with equality being the case $r = 0$). Since the runtime is dominated by the size of the lists, Eq.(8) is then minimised if we also choose $2^\ell \approx |L_1|$. By computing this for all $p \in [0, w]$, we eventually find the parameters for the minimal runtime.

Concerning the memory cost of the enumeration, we need enough space to hash the smaller of both initial lists and to save the matrices P_1, P_2 along with the transformed syndromes:

$$S_{\text{Enum}}(r, p, \ell, M) = \min\{|L_1|, |L_2|\} \cdot \ell + (n - k) \cdot (k + \ell + M).$$

A.3 DOOM With May-Meurer-Thomae (MMT)

The idea of MMT-ISD is to increase the possibilities of finding a desired sub-solution by introducing representations as sums $\mathbf{e}_2 = \mathbf{e}_{21} + \mathbf{e}_{22}$. The summands have the same lengths but lower weights, and are enumerated using a 2-list approach. Our DOOM variant is a generalisation of this which keeps the initial list sizes balanced. See Figure 7 for the splitting of $\mathbf{e}_2$.

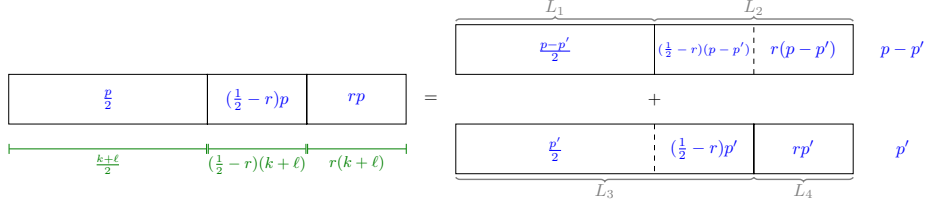


Fig. 7: Distribution of the length (green) and weight (blue) in the representations of $\mathbf{e}_2$ in MMT-DOOM

This allows us to transform the equation for the sub-solutions to

$$P_2(\mathbf{e}_{211} 0)^T + P_2(0 \mathbf{e}_{212})^T = -P_2(\mathbf{e}_{221} 0)^T + \mathbf{s}_2^{(i)} - P_2(0 \mathbf{e}_{222})^T, \quad (9)$$

and fill four lists as follows:

$$\begin{aligned} L_1 &:= \left\{ P_2(\mathbf{e}_{211} 0)^T : \mathbf{e}_{211} \in \mathbb{F}_2^{\frac{k+l}{2}} \left(\frac{p-p'}{2} \right) \right\}, \\ L_2 &:= \left\{ P_2(0 \mathbf{e}_{212})^T : \mathbf{e}_{212} \in \mathbb{F}_2^{(\frac{1}{2}-r)(k+l)} \left(\left(\frac{1}{2}-r \right) (p-p') \right) \times \mathbb{F}_2^{r(k+l)} (r(p-p')) \right\}, \\ L_3 &:= \left\{ -P_2(\mathbf{e}_{221} 0)^T : \mathbf{e}_{221} \in \mathbb{F}_2^{\frac{k+l}{2}} \left(\frac{p'}{2} \right) \times \mathbb{F}_2^{(\frac{1}{2}-r)(k+l)} \left(\left(\frac{1}{2}-r \right) p' \right) \right\}, \\ L_4 &:= \left\{ \mathbf{s}_2^{(i)} - P_2(0 \mathbf{e}_{222})^T : i \in \{1, \dots, M\}, \mathbf{e}_{222} \in \mathbb{F}_2^{r(k+l)} (rp') \right\}. \end{aligned}$$

The parameters $r \in [0, 1/2]$ and $p' \leq p/2$ are used to balance the sizes of these initial lists. Constructing these initial lists has cost analogous to that of Section A.2.

The potential sub-solution $\mathbf{e}_2$ is then constructed in two steps. First, we match the elements of pairs of neighbouring lists via hash joining. Unlike before, we do not match on all ℓ entries at this step, but just on the first $b \leq \ell$ entries:

$$\begin{aligned} L_{12} &:= L_1 \bowtie_b L_2 := \{ P_2 \mathbf{e}_{21} \in L_1 + L_2 : \pi_b(P_2 \mathbf{e}_{21}) = \mathbf{0}^b \}, \\ L_{34} &:= L_3 \bowtie_b L_4 := \{ \mathbf{s}_2^{(i)} - P_2 \mathbf{e}_{22} \in L_3 + L_4 : \pi_b(\mathbf{s}_2^{(i)} - P_2 \mathbf{e}_{22}) = \mathbf{0}^b \}, \end{aligned}$$

where $\pi_b: \mathbb{F}_2^\ell \rightarrow \mathbb{F}_2^b$ is the projection onto the first b entries. So now all pairs of elements of these lists fulfil Eq.(9) in the first b entries. This step costs $(|L_{12}| + |L_{34}|) \cdot (k + \ell)$ in time, as only vector additions are performed.

The second step consists of matching these elements on the remaining $\ell - b$ entries. We achieve this by simply performing a hash join $L_{12} \bowtie L_{34}$. As in DS-DOOM, we can check during the construction of $L := L_{12} \bowtie L_{34}$, whether a sub-solution also yields a solution to the original problem. So the cost in time is again $|L| \cdot ((k + \ell) + (n - k - \ell) \cdot (k + \ell))$.

Expected Cost and Parameter Choice. The algorithm will be successful if at least one error vector is well-permuted and any of their representations reaches the lists L_{12} and L_{34} . Here, well-permuted means that $w - p$ non-zero entries are distributed in the first $n - k - \ell$ entries, and the remaining p non-zero entries are distributed in the last $k + \ell$ entries according to (the left-hand side of) Figure 7. For a fixed error vector this happens with probability

$$\alpha_{p,\ell,r} := \frac{\binom{n-k-\ell}{w-p} \binom{(k+\ell)/2}{p/2} \binom{(1/2-r)(k+\ell)}{(1/2-r)p} \binom{r(k+\ell)}{rp}}{\binom{n}{w}}.$$

Now, for a well-permuted error vector, the number of representations across the initial lists is given by

$$R(p, p', r) := \binom{p/2}{p'/2} \binom{(1/2-r)p}{(1/2-r)p'} \binom{rp}{rp'},$$

and the probability that a fixed representation reaches the lists L_{12} and L_{34} is 2^{-b} . Analogous to [MMT11, Theorem 2], one can show that the probability that at least one representation of any well-permuted error vector reaches the lists L_{12} and L_{34} is at least $1/2$ if one chooses

$$b \leq \begin{cases} \log_2(\tilde{M} \cdot R(p, p', r)) - 2 & , \text{ for } M = 1 \\ \log_2(\tilde{M} \cdot R(p, p', r)) - 1 & , \text{ for } M > 1 \end{cases},$$

where $\tilde{M} := \max\{1, M\alpha_{p,\ell,r}\}$ is the expected number of well-permuted error vectors, given at least one.

Analogous to DS-DOOM, the probability that at least one error vector is well-permuted can then be safely approximated by

$$\epsilon_{p,\ell,r,M} \approx \min\left\{1, \frac{M\alpha_{p,\ell,r}}{2}\right\}.$$

The expected runtime of the enumeration step is again dominated by the sum of the list sizes, which are the following:

$$\begin{aligned} |L_1| &= \binom{(k+\ell)/2}{(p-p')/2}, & |L_2| &= \binom{(1/2-r)(k+\ell)}{(1/2-r)(p-p')} \binom{r(k+\ell)}{r(p-p')}, \\ |L_3| &= \binom{(k+\ell)/2}{p'/2} \binom{(1/2-r)(k+\ell)}{(1/2-r)p'}, & |L_4| &= M \cdot \binom{r(k+\ell)}{rp'}, \\ \mathbb{E}|L_{12}| &= |L_1| \cdot |L_2| \cdot 2^{-b}, & \mathbb{E}|L_{34}| &= |L_3| \cdot |L_4| \cdot 2^{-b}, \\ \mathbb{E}|L| &= |L_{12}| \cdot |L_{34}| \cdot 2^{-(\ell-b)}, \\ &\approx |L_1| \cdot |L_2| \cdot |L_3| \cdot |L_4| \cdot 2^{-\ell-b}. \end{aligned}$$

Note that the sizes of the last three lists is minimised if we choose b maximally. To have balanced initial lists, we need that

$$M \leq \sqrt[3]{\left(\binom{(k+\ell)/2}{(p-p')/2}\right)^2 \left(\binom{(k+\ell)/2}{p'/2}\right)^2} = \left(\binom{(k+\ell)/2}{(p-p')/2} \binom{(k+\ell)/2}{p'/2}\right)^{\frac{2}{3}},$$

with equality occurring (asymptotically) if L_4 only contains the syndromes (i.e. $r = 0$), and p' is chosen such that the first three lists are balanced. Under this condition, we then compute the parameters p' and r as follows:

1. Assuming that L_3 and L_4 are balanced, compute p' that minimises the difference of their asymptotic size to the first two lists:

$$p' := \arg \min_{0 \leq p' \leq p/2} \left| \binom{(k+\ell)/2}{(p-p')/2} - \sqrt{M \cdot \binom{k+\ell}{p'}} \right|.$$

2. With this p' , compute the distribution factor r that actually balances the list sizes of L_3 and L_4 :

$$r := \arg \min_{0 \leq r \leq 1/2} \left| \binom{(k+\ell)/2}{p'/2} \binom{(1/2-r)(k+\ell)}{(1/2-r)p'} - M \cdot \binom{r(k+\ell)}{rp'} \right|.$$

Concerning the memory cost for the enumeration, a streaming implementation as described in [EMZ22] should be used. The memory cost up to constant factors will then be

$$\ell \cdot (\max\{\min\{|L_1|, |L_2|\}, \min\{|L_3|, |L_4|\}\} + \min\{|L_{12}|, |L_{34}|\}),$$

in addition to the $(n-k)(k+\ell+M)$ bits needed to store the matrices P_1 , P_2 and the transformed syndromes.